

PROMPT LIBRARY

30+ ready-to-copy prompts for Claude Code

Copy any prompt below and paste it directly into your Claude Code session.

Getting Started

Understand a new codebase

Explore this codebase and give me a high-level overview: what it does, the tech stack, the folder structure, and the main entry points. Don't change any files.

Generate CLAUDE.md

```
/init
# Then review and customize the generated CLAUDE.md
```

Understand a specific file

Explain @src/lib/auth.ts in plain English. What does each function do, what are the edge cases, and what patterns does it use?

Find where something happens

Where in this codebase does user authentication happen? Show me the flow from login request to session creation. Don't modify anything.

Writing Code

Build a feature

Build a user registration form with email/password validation. Use our existing component patterns. Write tests. Don't touch unrelated files.

Add an API endpoint

Create a REST API endpoint POST /api/users that validates input with Zod, creates a user in the database, and returns 201. Add error handling and tests.

Implement from spec

Here's the spec: [paste spec]. Implement it step by step. After each step, run the tests. Stop and ask me if any test fails.

Scaffold a new module

Create a new module for payment processing with:

- Types in /src/types/payment.ts
- Service in /src/lib/payment-service.ts
- API route in /src/app/api/payments/route.ts
- Tests for the service

Debugging

Fix a bug

This error appears when I click submit: [paste error]. Find the root cause, explain why it happens, fix it, and add a test that would have caught it.

Debug a test failure

npm test fails on auth.test.ts with: [paste error]. Don't just make the test pass - find and fix the actual bug.

Trace a data flow

The checkout total shows \$0 for some users. Trace the data flow from cart items to the total calculation. Find where the bug is.

Performance issue

The /api/products endpoint takes 3+ seconds. Profile the code, identify bottlenecks, and suggest fixes. Don't change anything yet - just analyze.

Refactoring

Extract a reusable function

@src/components/Dashboard.tsx has duplicated data fetching logic. Extract it into a custom hook. Make sure all existing behavior is preserved. Run tests after.

CLAUDE CODE - Prompt Library (cont.)

Simplify complex code

@src/lib/pricing.ts is hard to follow. Refactor for readability:

- Break into smaller functions
- Add clear variable names
- Add JSDoc comments

Don't change the behavior. Run tests after each change.

Modernize old code

Update @src/utils/api-client.js to TypeScript:

- Add proper types
- Replace callbacks with async/await
- Keep the same public API
- Run tests to verify nothing breaks

Remove dead code

Find and remove all unused exports, unreachable code, and dead imports in /src/lib/. Use grep to verify nothing references them before deleting.

Testing

Write tests for a file

Write comprehensive tests for @src/lib/cart-service.ts.
Cover: happy path, edge cases, error cases. Use describe/it pattern.
Mock external dependencies. Run tests when done.

Increase test coverage

Run the test suite and identify files with low or no coverage.
Write tests for the 3 most critical untested files. Focus on business logic.

Write an integration test

Write an integration test for the checkout flow:

1. Add items to cart
2. Apply discount code
3. Process payment
4. Verify order is created

Mock only external APIs (payment gateway).

Fix flaky tests

These tests fail intermittently: [list]. Investigate why they're flaky.
Common causes: timing issues, shared state, external dependencies.
Fix them to be deterministic.

Code Review & Quality

Review my changes

Review all uncommitted changes (git diff). Focus on:

1. Logic errors and bugs
2. Security issues
3. Missing error handling

Don't comment on style. Give concrete fixes for each issue.

Security audit

Audit this codebase for security vulnerabilities:

- SQL/NoSQL injection
- XSS opportunities
- Auth bypass
- Exposed secrets
- Missing input validation

Prioritize by severity. Give specific fixes.

Pre-PR checklist

I'm about to open a PR. Check:

- All tests pass
- No console.logs left
- No hardcoded values
- Types are correct
- Error handling is complete
- New code has tests

Git & Workflow

CLAUDE CODE - Prompt Library (cont.)

Generate commit message

Look at my staged changes (`git diff --staged`) and generate a conventional commit message. Format: `type(scope): subject + body explaining WHY`.

Create a PR description

Based on all commits since main (`git log main..HEAD`), write a PR description:

- What changed and why
- How to test it
- Any breaking changes
- Screenshots needed?

Interactive rebase help

I have 12 messy commits on this branch. Help me interactive rebase to clean them into logical, atomic commits. Show me the rebase plan first.

Documentation

Generate README

Create a README.md for this project with:

- What it does (1 paragraph)
- Quick start (install + run)
- Environment variables needed
- Project structure overview
- How to contribute

Document an API

Generate API documentation for all routes in `/src/app/api/`.
For each endpoint: method, path, request body, response, error codes.
Format as Markdown.

Add inline docs

Add JSDoc comments to all exported functions in `@src/lib/`.
Include: description, `@param` types, `@returns`, `@throws`, `@example`.